

ID: 14661

Name: Ridaa Bilal

Section: D

Department: BS-SE

CLASS TASK

Depth-First Search (DFS)

- DFS stands for **Depth-First Search**.
- It explores as **deep as possible** along each branch before backtracking.
- DFS uses a **stack** data structure (can be implemented using recursion or explicitly).
- It is useful for problems like **pathfinding, topological sorting, and cycle detection**.
- DFS may not always find the shortest path in unweighted graphs.

```
n [2]: ▶ def dfs(graph, node, visited):
        if node not in visited:
            print(node, end=" ")
            visited.add(node)
            for neighbor in graph[node]:
                dfs(graph, neighbor, visited)

        graph = {
            'P': ['Q', 'R'],
            'Q': ['S', 'T'],
            'R': ['U'],
            'S': ['V'],
            'T': ['W'],
            'U': ['X'],
            'V': [],
            'W': [],
            'X': []
        }

        print("DFS Traversal:")
        dfs(graph, 'P', set())
```

```
DFS Traversal:
P Q S V T W R U X
```

Breadth-First Search (BFS)

- BFS stands for **Breadth-First Search**.
- It explores all **neighbors level by level**, starting from the source node.
- BFS uses a **queue** data structure to maintain the order of exploration.
- It is useful for finding the **shortest path** in an **unweighted graph**.
- BFS guarantees visiting the closest nodes first before moving deeper.

```
In [3]: ➤ def bfs(graph, start):
    visited = set()
    queue = [start]

    while queue:
        node = queue.pop(0)
        if node not in visited:
            print(node, end=" ")
            visited.add(node)
            for neighbor in graph[node]:
                queue.append(neighbor)

graph = {
    'P': ['Q', 'R'],
    'Q': ['S', 'T'],
    'R': ['U'],
    'S': ['V'],
    'T': ['W'],
    'U': ['X'],
    'V': [],
    'W': [],
    'X': []
}

print("\nBFS Traversal:")
bfs(graph, 'P')
```

```
BFS Traversal:
P Q R S T U V W X
```